



Aqsa Khalid
Lecturer
School of Computing

CS3002 Information Security



Security Design Principles and Security Planning

Security Design Principles



- When designing (or configuring) an information system, following these **generic** guidelines would greatly reduce the security vulnerabilities.
 - Following these patterns should help you avoid lots (but not all) of common security problems.
- Some of these principles were initially proposed by J. Saltzer and M. Schroeder in “The Protection of Information in Computer Systems” (1975). Other principles have been proposed by experienced security professionals.

Security Design Principles

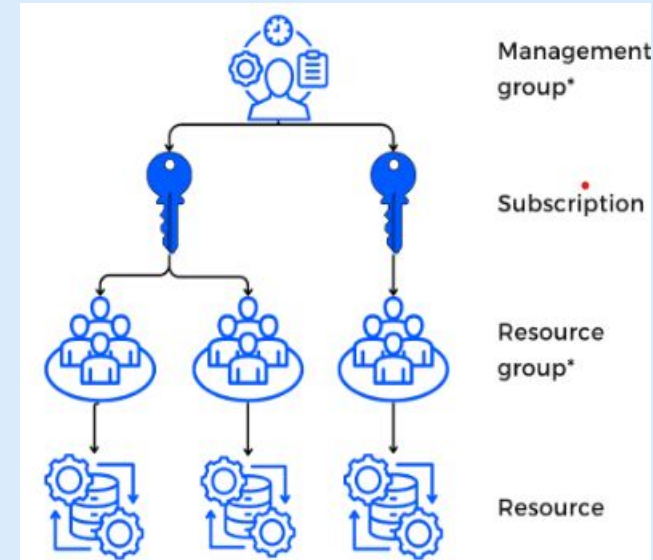


1. Least Privilege
2. Separation of privilege
3. Fail-safe defaults
4. Complete mediation
5. Open design
6. Economy of mechanism
7. Psychological acceptability
8. Defense in Depth (Layering)
9. Compartmentalization / Segmentation

Least privilege



- Provide **bare minimum privileges** to a program or user to function properly
- Temporary elevation should be relinquished immediately
- Granularity of privileges



Advantage

- Abuse of privileges is restricted
- Damage caused by the compromised user or application is reduced

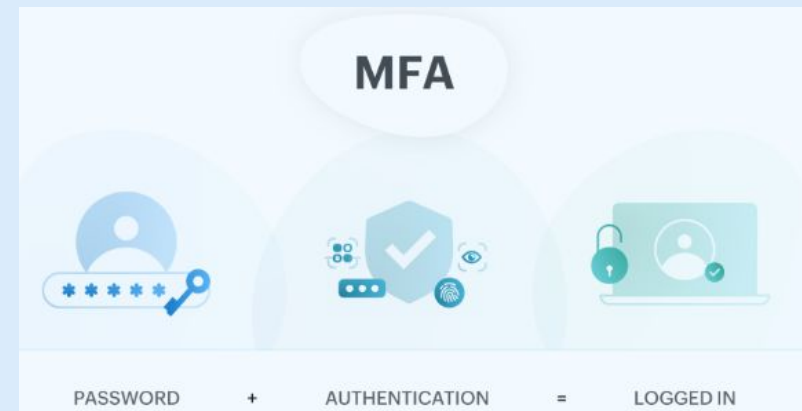
Separation of Privilege



- The **access** should **not be granted** based on **single condition**
- Multiple conditions should be required to achieve access to restricted resources
e.g. Banking apps

Examples:

- Two persons to sign cheque
- Password login + OTP to perform financial transactions



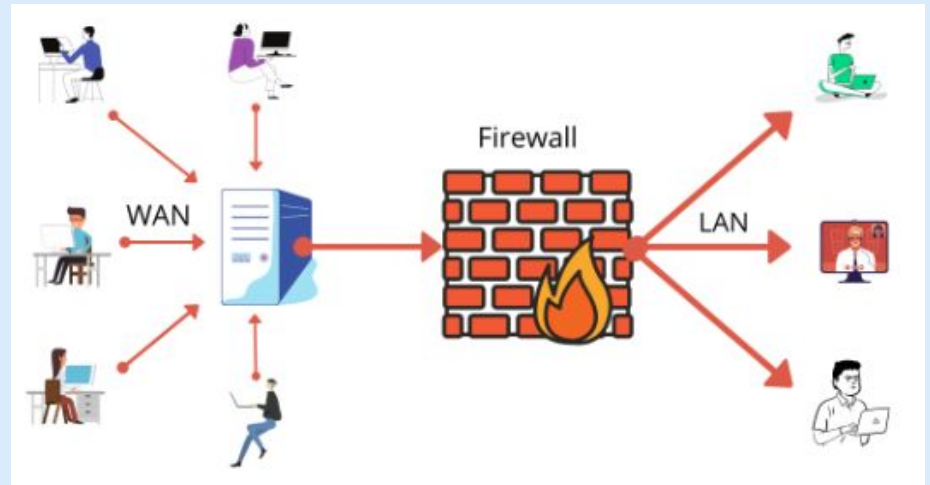
Fail-safe defaults



- The default configuration of a system should have a conservative approach...
 - Default access to an object or resource is none
 - Explicit access to an object/resource should be given

Examples

- Access Control Lists
- Firewall rules



Complete mediation



- Instead of one-time check, **every access** to a resource **must be checked** for compliance with a protection scheme
- Do not rely on caching of access information
- Security vs performance dilemma

Open design



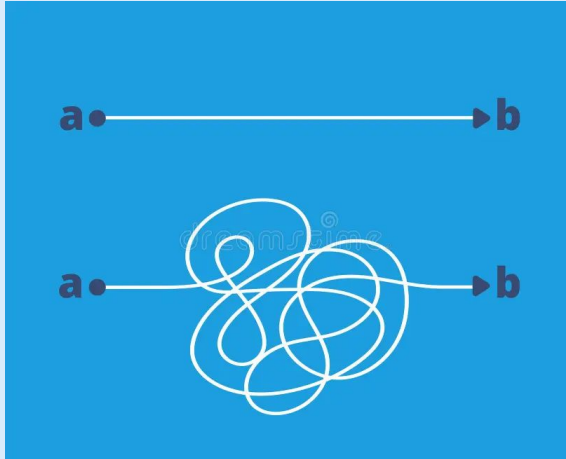
- Design of a **security mechanism** should be **open** rather than **secret**
- Open design can be **reviewed by** many **experts**, their feedback helps in improving it.

For example:

- Cryptographic algorithm
- Open Source Software
- Bug Bounty Programs

Economy of mechanism



- Simplicity in design and implementation of security measures
- 
- The diagram consists of two horizontal paths on a blue background. The top path is a straight white line with a dot labeled 'a' at the start and an arrow pointing to a dot labeled 'b' at the end. The bottom path starts with a dot labeled 'a', followed by a highly convoluted, tangled white line that loops and crosses itself multiple times, before ending with an arrow pointing to a dot labeled 'b'.
- A simple secure framework provides...
 - Fewer errors
 - Development, testing and verification of security measures is easy
 - Less assumptions

Psychological acceptability



- **Ease of Access:**

Security mechanism should not make the resources difficult to access

- **User–Friendly Interface:**

User interface should be well designed and intuitive

- **User Expectations:**

Security related setting should consider the expectation of ordinary users

Defense in Depth (Layering)



- Use multiple, overlapping protection approaches addressing the people, technology, and operational aspects of information systems.
- Failure or circumvention of any individual protection approach will not leave the system unprotected. Layering provides multiple barriers between an adversary and protected information or services.
- Software bugs are inevitable, so a redundancy in protection is a highly desirable feature.

Defense in Depth (Layering)



- Example: A database server in a datacenter may have these layers of protection
 - Physical security (Restricted entry, CCTV, locked racks)
 - Firewalls: monitoring traffic based on IP/port
 - Intrusion Prevention System: monitoring for suspicious network activity
 - Antimalware software
 - Access control policies defined
 - Encryption at rest

Compartmentalization / Segmentation



- Segment data and networks into distinct zones. By erecting boundaries, a breach in one compartment cannot easily spread to the rest of the system, containing damage.
- This principle helps in minimizing the impact of an attack (so called **blast radius**) in case of a successful breach.

Compartmentalization / Segmentation



- Examples
 - Processes and files of individual users on a computer are isolated from those of other users, unless explicitly shared.
 - Cloud servers host each customer's files and data in separate virtual machines. Virtualization software enforces the isolation boundaries.
 - Corporate networks place public facing servers (web, email) on a different network segment, separated from internal private network using firewalls

Caveats



- No list of principles like the one above is ever perfect.
- Not only do our principles present an incomplete picture, but they also sometimes conflict with each other. As with any complex set of principles, there are often subtle tradeoffs involved. For example, the defense-in-depth principle suggests that you build redundancy into your systems, whereas the simplicity principle advises that you avoid unnecessary code.
- Therefore, application of these principles must be sensitive to context, taking into account risk parameters (how likely is a given attack, how much damage it will cause).

References



- Stallings SPP, chap 1
- Whitman & Mattord, chap 1–2
- John Viega & Gary McGraw, “Building Secure Software”, Chapter 5
- Kohnfelder, “Designing Secure Software”, Chapter 4
- <https://devguide.owasp.org/en/02-foundations/03-security-principles/>